Intro to Deep Learning

Lecture 08: Attention and Transformers

Stanislav Don

DS212 - Intro to Deep Learning

Today

By the end of the lecture, you should be able to answer:

1. What problem does attention solve?
2. What are tokens and embeddings?
3. What are queries, keys, and values?
4. How does scaled dot-product attention work?
5. Why do LLMs need causal masking?
6. What is inside a transformer block?

Lecture rhythm: tokens -> Q/K/V -> attention weights -> masks -> transformer blocks

Recap From Lecture 07

Pretrained CNNs reuse visual representations.

Today we move to architectures that power:

- language models
- vision transformers
- CLIP-like multimodal systems
- modern generative models

The central operation:

```
attention
```

Part 1

Why attention?

Why Attention Became Important

Many inputs are sequences or sets:

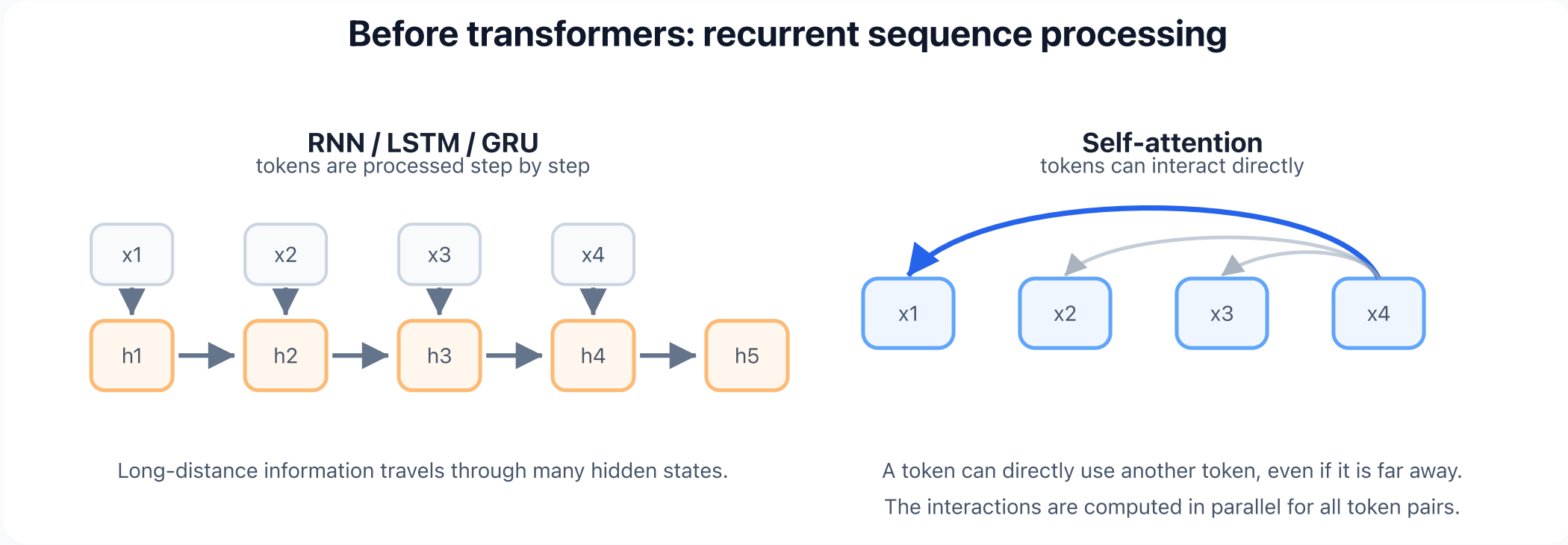
- words in a sentence
- patches in an image
- frames in a video
- objects in a scene

The model needs to decide:

```
which parts of the input should influence each other?
```

Attention makes this interaction data-dependent.

Before Transformers: Sequential Models



RNNs, LSTMs, and GRUs process tokens through a hidden-state chain.

Attention gives tokens more direct access to each other and is easier to parallelize.

Fixed Mixing vs Content-Based Mixing

In a CNN:

```
nearby pixels interact through fixed local neighborhoods
```

In attention:

```
each token can choose which other tokens to use
```

This choice depends on the token content.

That is the key shift.

Tokens

A token is one unit of input.

For text:

```
words, subwords, or characters
```

For images:

```
patches
```

For a sequence of length `T`, we have `T` tokens.

Embeddings

A token is represented by a vector.

This vector is called an embedding.

Shape convention:

```
X: [B, T, D]
```

Meaning:

```
B = batch size
T = number of tokens
D = embedding dimension
```

Example Shape

For example:

```
X.shape = [1, 4, 6]
```

Meaning:

```
1 sequence
4 tokens
6 numbers per token
```

Attention returns a new representation for each token.

Attention Intuition

For each token, attention asks:

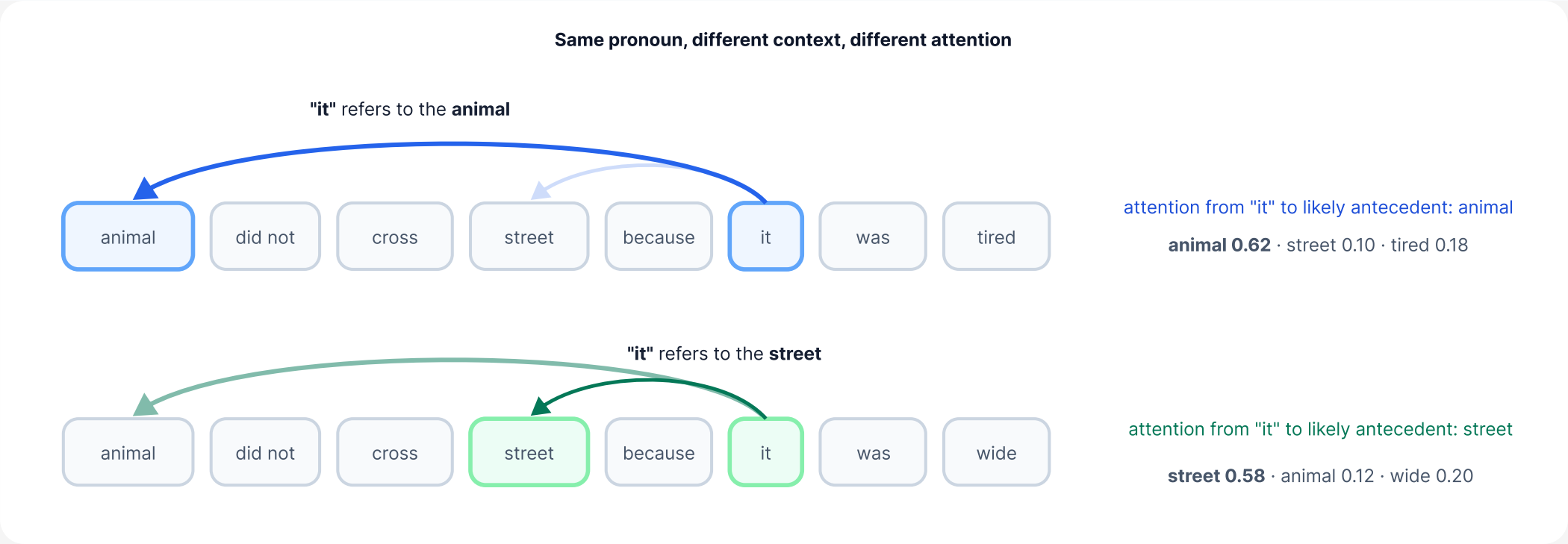
```
which other tokens are relevant to me?
```

Then it builds:

```
a weighted mixture of information from those tokens
```

The weights are learned indirectly through Q, K, and V projections.

Text Example: What Words Look At



Attention can make the same token use different context depending on meaning.

For a pronoun like `it`, the useful context may be a different earlier word in each sentence.

Part 2

Q, K, V and attention weights

Query, Key, Value

For each token embedding, we create three vectors:

```
query: what am I looking for?
key:   what do I contain?
value: what information do I provide?
```

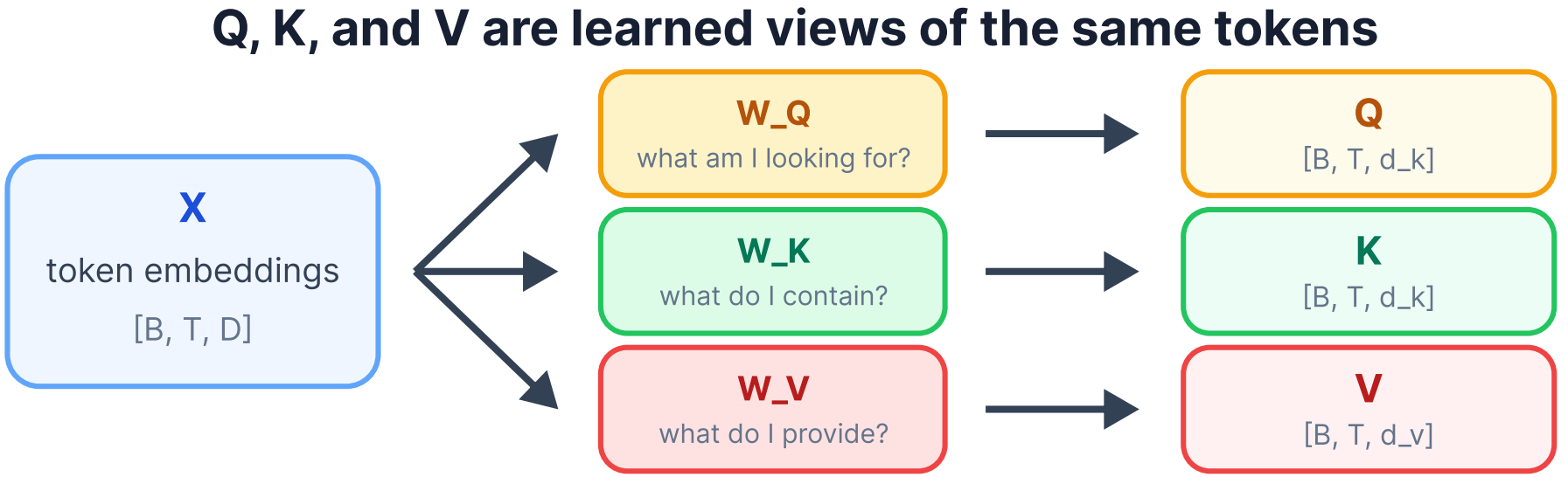
These come from learned linear projections:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q$$
$$\mathbf{K} = \mathbf{X}\mathbf{W}_K$$
$$\mathbf{V} = \mathbf{X}\mathbf{W}_V$$

Shapes:

$$\mathbf{X} \in \mathbb{R}^{B \times T \times D}$$
$$\mathbf{W}_Q, \mathbf{W}_K \in \mathbb{R}^{D \times d_k}, \quad \mathbf{W}_V \in \mathbb{R}^{D \times d_v}$$

Q, K, V Projections



Attention Scores

To compare queries and keys, use dot products:

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^\top$$

For each query token, we compare it with every key token.

Shape:

$$Q \in \mathbb{R}^{B\times T\times d_k}, \quad K^\top \in \mathbb{R}^{B\times d_k\times T}$$

$$S \in \mathbb{R}^{B\times T\times T}$$

What Does scores[i, j] Mean?

For one sequence:

```
scores[i, j]
```

means:

```
how strongly token i attends to token j
before softmax
```

Rows are query positions. Columns are key positions.

This is why attention heatmaps use:

```
y-axis = query position
x-axis = key position
```

Mini-Check: Silent Attention Bug

A student implements attention and the code runs:

```
scores = Q.transpose(-2, -1) @ K
weights = torch.softmax(scores, dim=-1)
output = weights @ V.transpose(-2, -1)
```

The output shape can be forced to look plausible later with reshaping.

But the attention heatmap has shape:

```
[d_k, d_k] instead of [T, T]
```

What relationship is the model learning here?

What quick shape assertion would catch this bug?

Answer

The model is comparing **feature dimensions**, not tokens.

This line:

```
scores = Q.transpose(-2, -1) @ K
```

produces:

```
[B, d_k, d_k]
```

Correct attention scores should compare every query token with every key token:

```
scores = Q @ K.transpose(-2, -1)
assert scores.shape == (B, T, T)
```

Why Scale By sqrt(d_k)?

Dot products can become large when `d_k` is large.

Large scores can make softmax too sharp:

```
one token gets almost all probability
gradients become less useful
```

So we scale:

$$\mathbf{S} = \frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}$$

This keeps attention weights better behaved.

Softmax Over Keys

After scaling, apply softmax:

```
weights = torch.softmax(scores, dim=-1)
```

`dim=-1` means:

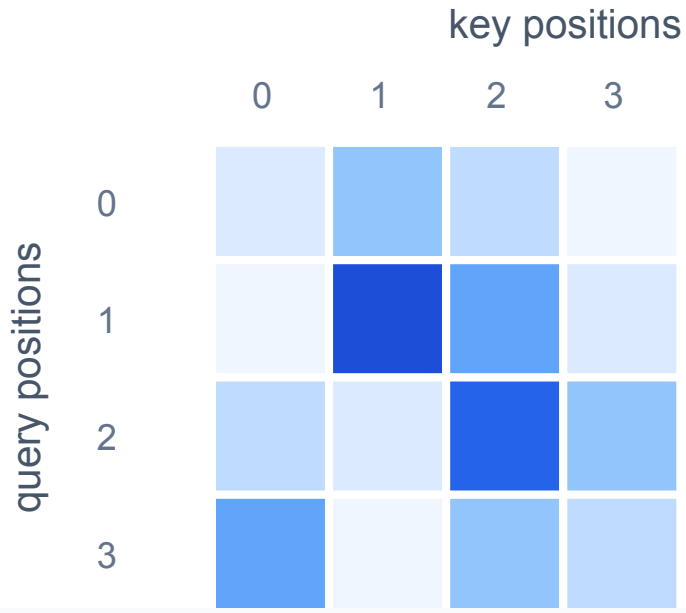
```
softmax across keys for each query
```

Each row sums to 1:

```
attention weights are a distribution over source tokens
```

Attention Heatmap

Attention weights are row-wise distributions over keys



one row = one query

softmax over keys

row sum = 1

Weighted Sum Of Values

The final attention output is:

$$\mathbf{O} = \mathbf{A}\mathbf{V}$$

where $\mathbf{A}$ is the attention-weight matrix.

Shape:

```
A:        [B, T, T]
V:        [B, T, d_v]
O:        [B, T, d_v]
```

Each output token becomes:

```
a weighted mixture of value vectors
```

Scaled Dot-Product Attention

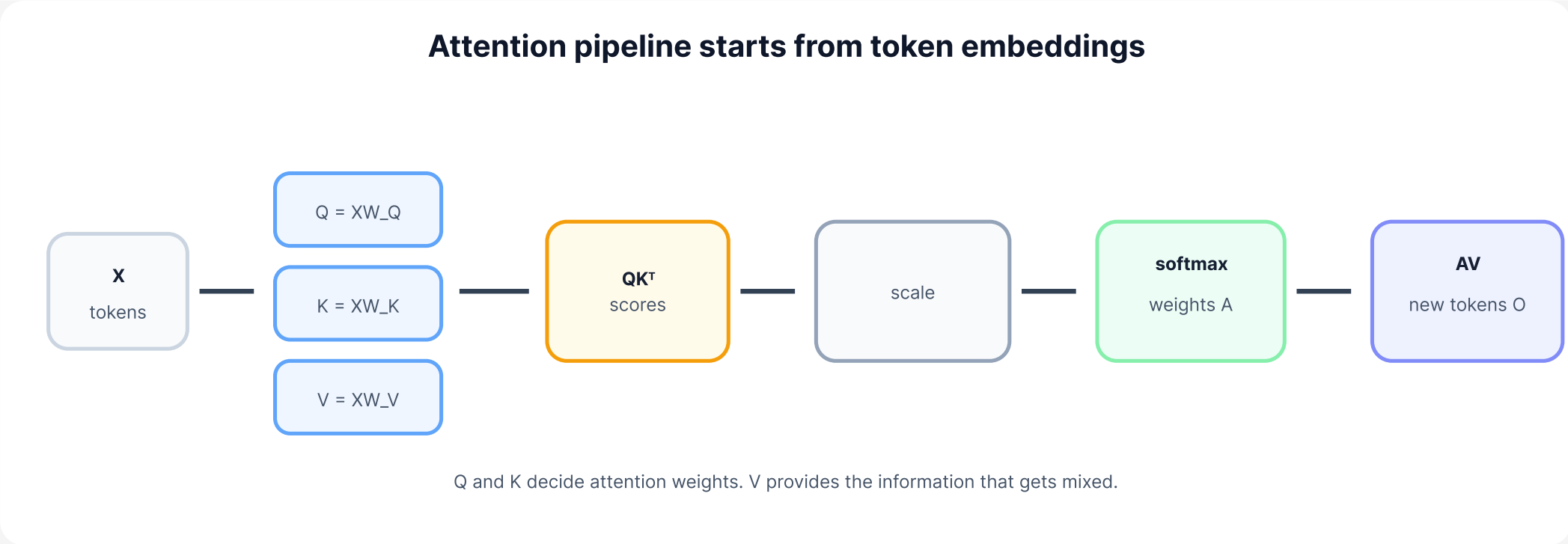
The full operation:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}$$

Pipeline:

```
scores -> scale -> softmax -> weighted sum of values
```

Attention Pipeline



First project token embeddings into `Q`, `K`, and `V`.

Then compute token-to-token weights and use them to mix value vectors.

Single-Head Self-Attention

Self-attention means:

```
Q, K, and V all come from the same input X
```

Single-head means:

```
we do this attention operation once
```

Multi-head attention repeats this idea several times in parallel.

Why "Self" Attention?

The sequence attends to itself.

Example:

```
each word token can use information from other word tokens
```

For images:

```
each patch token can use information from other patch tokens
```

The model learns which interactions matter.

Multi-Head Attention

Single-head attention gives one type of relationship.

Multi-head attention lets the model learn several relationships at once.

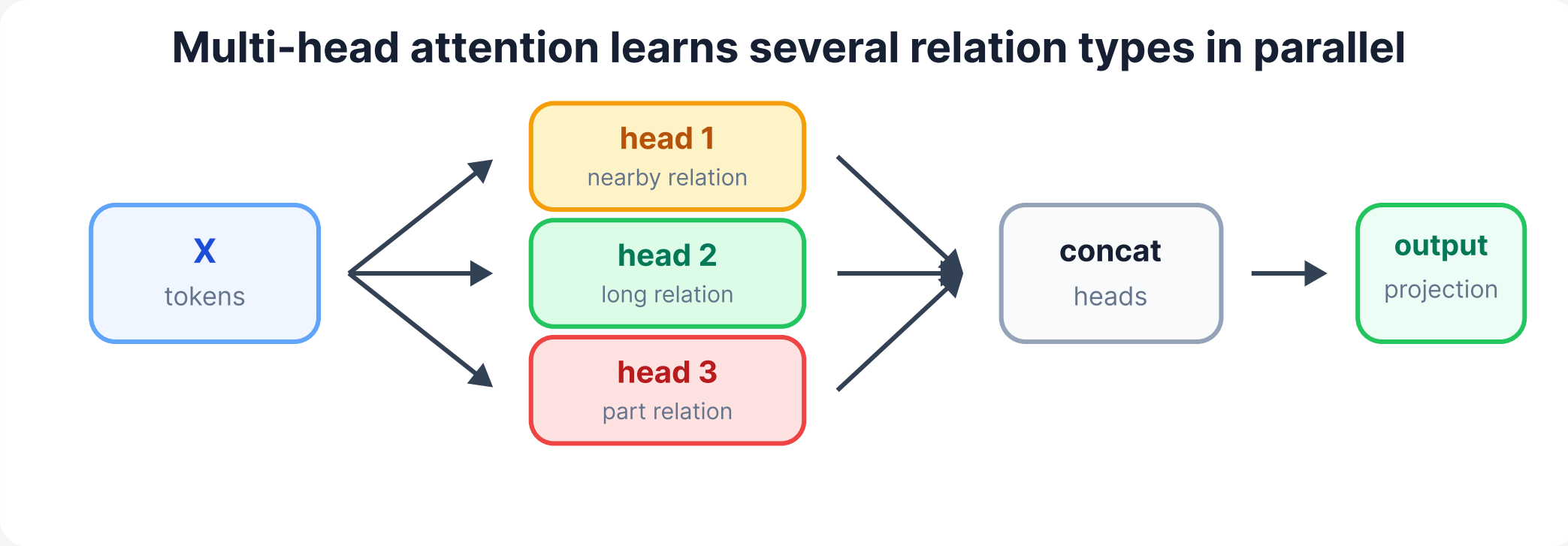
Examples:

- nearby token relation
- long-distance relation
- object-part relation
- syntax-like relation

Basic idea:

```
run several attention heads in parallel
then combine their outputs
```

Multi-Head Attention: Picture



Part 3

Causal masking

Causal Masking

Autoregressive language models predict the next token.

When predicting token `t`, the model may use:

```
tokens 1, 2, ..., t
```

It must not use:

```
tokens t+1, t+2, ...
```

Otherwise training would leak the answer.

Causal Mask Shape

For `T = 4`, causal masking blocks future positions:

```
allowed:
token 0 attends to 0
token 1 attends to 0, 1
token 2 attends to 0, 1, 2
token 3 attends to 0, 1, 2, 3
```

The blocked region is the upper triangle of the `[T, T]` score matrix.

Mask Before Softmax

Masking must happen before softmax.

Correct:

```
scores -> mask future scores to very negative -> softmax
```

Wrong:

```
scores -> softmax -> zero future probabilities
```

If we mask after softmax, rows may no longer sum to 1.

Mini-Check: Future Leakage

You train a next-token model.

Symptoms:

- training loss becomes suspiciously low
- generated text is still poor
- attention maps show nonzero weights above the diagonal

What is the likely bug?

Where in the attention pipeline would you inspect first?

Answer

Likely bug:

```
causal mask is missing, inverted, or applied after softmax
```

Inspect the score matrix **before softmax**.

Future positions should be set to a very negative value:

```
scores = scores.masked_fill(mask, -1e9)
weights = torch.softmax(scores, dim=-1)
```

The final attention heatmap should have near-zero probability above the diagonal.

Why Very Negative?

In code:

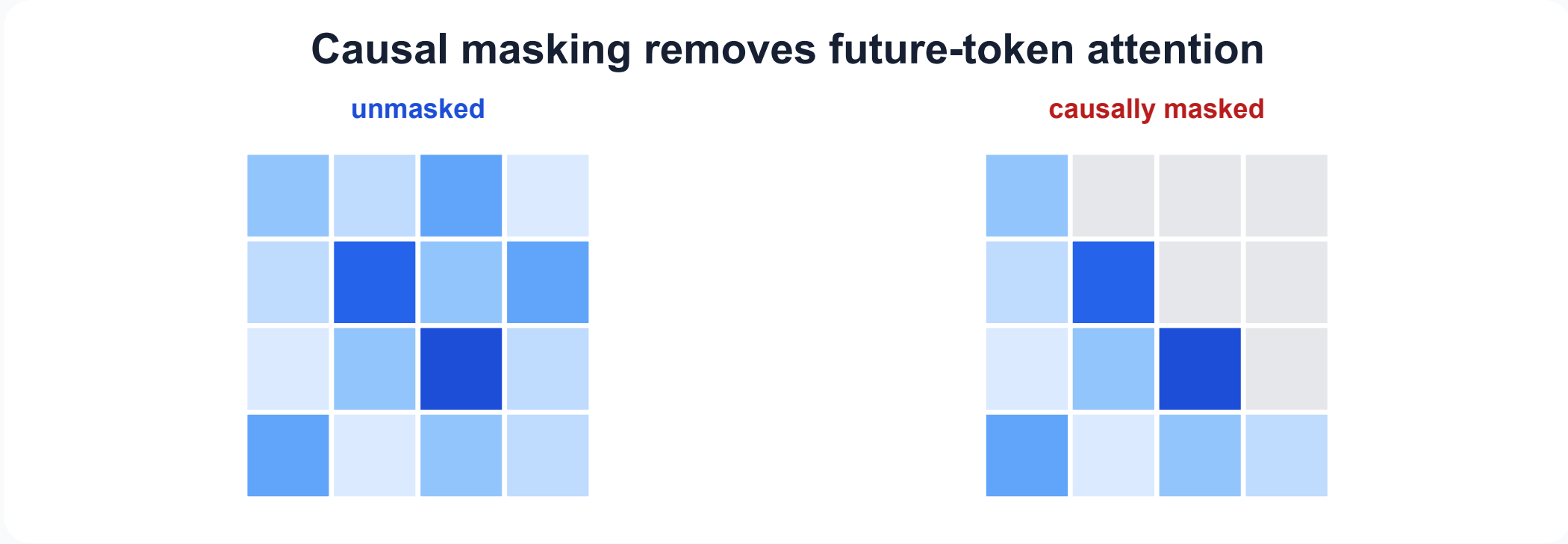
```
scores = scores.masked_fill(mask, -1e9)
weights = torch.softmax(scores, dim=-1)
```

Softmax of a very negative number is almost zero.

So future positions receive almost no attention probability.

This is the practical trick students will implement.

Causal Mask Heatmap



Part 4

Transformer blocks

Transformer Block

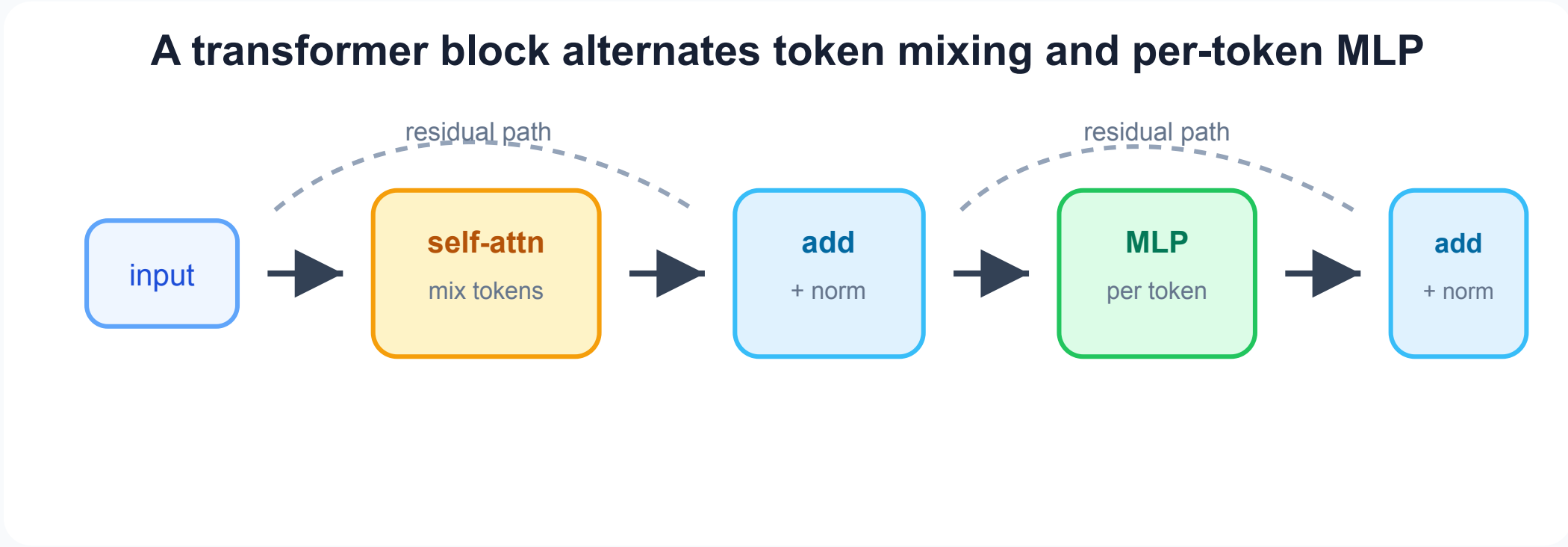
A transformer block combines:

```
self-attention
residual connection
layer normalization
MLP / feed-forward network
```

High-level structure:

```
attention mixes information across tokens
MLP processes each token representation
residuals help optimization
normalization stabilizes training
```

Transformer Block: Picture



Residuals Return

From ResNet:

```
y = F(x) + x
```

Transformers also use residual connections.

Why?

```
preserve information
make deep networks easier to train
give gradients direct paths
```

This is one reason the ResNet idea matters beyond CNNs.

Layer Normalization

LayerNorm normalizes within each token representation.

Unlike BatchNorm:

```
BatchNorm depends on batch statistics
LayerNorm works per example/token
```

This is useful for sequence models where batch sizes and sequence lengths vary.

For this lecture, remember:

```
LayerNorm stabilizes transformer training
```

Feed-Forward Network

The transformer MLP is applied to each token independently.

Typical pattern:

```
Linear -> activation -> Linear
```

Attention mixes information across tokens.

The MLP transforms each token's representation after mixing.

Both parts are needed.

Part 5

Images as tokens

Text Tokens To Image Patches

Attention is not only for text.

Vision Transformers use image patches as tokens.

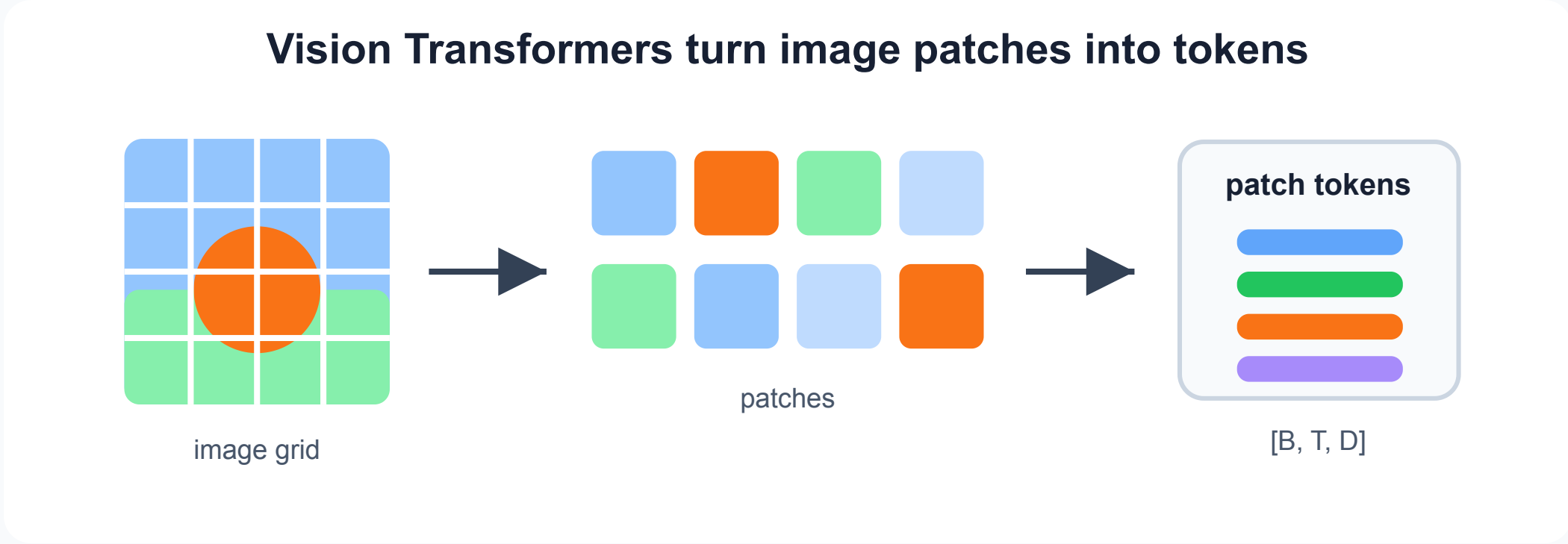
Example:

```
image:  [1, 3, 32, 32]
patch:  8 x 8
tokens: 16 patches
```

Then:

```
patches -> embeddings -> self-attention
```

Image Patches As Tokens



Common Implementation Mistakes

Mistake 1:

```
mixing up [B, T, D] shapes
```

Mistake 2:

```
softmax over the wrong dimension
```

Mistake 3:

```
forgetting division by sqrt(d_k)
```

Mistake 4:

```
applying the causal mask after softmax
```

Debugging Checklist

For self-attention, check:

- `Q`, `K`, `V` shapes
- score shape is `[B, T, T]`
- softmax uses `dim=-1`
- attention rows sum to 1
- mask is applied before softmax
- output shape is `[B, T, d_v]`

Shapes will catch most bugs.

Seminar Bridge

In Seminar 08, you will:

1. create a tiny token embedding tensor
2. compute Q, K, and V
3. compute attention scores and weights by hand
4. visualize attention as a heatmap
5. implement single-head self-attention
6. add a causal mask
7. turn image patches into tokens

Main goal:

```
understand the attention pipeline end to end
```

Takeaways

Attention lets tokens exchange information based on content.

Key ideas:

- Q asks, K matches, V provides information
- attention weights are row-wise softmax probabilities
- scaling by `sqrt(d_k)` stabilizes softmax
- causal masks prevent future-token leakage
- transformer blocks combine attention, MLPs, norms, and residuals

Next lecture:

```
embeddings, metric learning, and visual search
```